

立即体验

趁热体验, 免费的Claude 4正式上线Trae海外版。

AI 原生 IDE

AI 协作

上下文理解

Swift最新经典面试题（含答案）

无知的前端

2025-03-27

155

阅读24分钟

专栏: Swift

趁热体验, 免费的Claude 4正式上线Trae海外版。

立即体验

后续持续更新

1、介绍一下 Swift?

Swift是苹果在2014年6月WWDC发布的全新编程语言, 借鉴了JS,Python,C#,Ruby等语言特性,看上去偏脚本化,Swift 仍支持 [cocoa touch](#) 框架

他的优点:

1. Swift更加安全, 它是类型安全的语言。
2. Swift容易阅读, 语法和文件结构简化。
3. Swift更易于维护, 文件分离后结构更清晰。
4. Swift代码更少, 简洁的语法, 可以省去大量冗余代码
5. Swift速度更快, 运算性能更高。

2、Swift 中类(class) 和 结构体(struct) 的区别,以及各自优缺点?

- 类:
- • 引用类型
- •

在进行变量赋值时, 是通过指针copy,属于浅拷贝(shallow copy)

数据的存储是在堆空间
- • 可以被继承(前提是类没有被 final 关键字修饰), 子类可以使用父类的属性和方法
- • (当class继承自 Object,拥有runtime机制)类型转换可以在运行时检查和解释一个实例对象
- • 用 deinit (析构函数) 来释放资源 类似OC (dealloc)
- • 类的方法地址 是不确定的, 只有在具体运行时, 才能确定调用的具体值
- 结构体
- • 值类型
- •

在进行变量赋值是, 是深拷贝(deep copy), 产生了新的副本

数据的存储时在栈空间(大部分情况下, 不需要考虑内存泄露问题, 栈空间的特点是用完即释放)(注: 如果结构体的属性包含引用类型(例如另一个类的实例), 那么这些引用类型本身将存储在堆内存中。但是, 结构体中的这些引用(即指针)仍然会存储在栈内存中; 虽然结构体本身是值类型, 但它可以包含指向堆上数据的引用)
- • 结构体调用方法, 在编译完成就可以确定方法具体的地址值, 以便直接调用

综上, 在满足程序要求的情况下 优先使用 结构体

3、Swift中strong 、weak和unowned是什么意思? 二者有什么不同? 何时

在

无知的前端

iOS工程师

榜上有名

33

文章

8.0k

阅读

23

粉丝

关注

私信

目录

收起

- 1、介绍一下 Swift?
- 2、Swift 中 类(class) 和 结构体(struc...
- 3、Swift中strong 、weak和unowned...
- 4、访问控制关键字 open, public, inte...
- 5、Swift 和OC 如何相互调用?
- 6、类(class) 和 结构体(struct) 有什么...
- 7、class 和 struct 比较,优缺点?
- 8、copy - on - write 或者 理解Swift...
- 9、Swift 为什么将 Array、String、Di...

相关推荐

- OC项目转Swift指南
- 15k阅读 · 82点赞
- 答“卓同学的 Swift 面试题”--下篇
- 1.5k阅读 · 5点赞
- iOS-Swift 独孤九剑：十三、面向协议...
- 3.8k阅读 · 36点赞
- iOS最新面试题Objective - C部分（含...
- 110阅读 · 1点赞
- iOS CocoaPods专栏（一）—— 简介、...
- 2.6k阅读 · 6点赞

精选内容

- 6月最后一周, 程序员如何用3天换NVI...
- 程序员飞鱼 · 38阅读 · 4点赞
- SwiftUI 利用本机 “AI” 让 App 懂你的喜...
- 大熊猫侯佩 · 9阅读 · 0点赞
- 面试题大杂烩(三)🤖=>数组与对象扁平...
- Danta · 19阅读 · 5点赞
- 递归: JavaScript中的强大工具
- 前端小巷子 · 18阅读 · 0点赞
- SwiftUI-3D模型加载与显示

使用unowned?

YungFan · 20阅读 · 1点赞

Swift 的内存管理机制与 Objective-C一样为 ARC（Automatic Reference Counting）。它的基本原理是，一个对象在没有任何强引用指向它时，其占用的内存会被回收。反之，只要有任何一个强引用指向该对象，它就会一直存在于内存中。

找对属于你的技术圈子
回复「进群」加入官方微信群



- **strong** 代表着强引用，是默认属性。当一个对象被声明为 **strong** 时，就表示父层级对该对象有一个强引用的指向。此时该对象的引用计数会增加1。
- **weak** 代表着弱引用。当对象被声明为 **weak** 时，父层级对此对象没有指向，该对象的引用计数不会增加1。它在对象释放后弱引用也随即消失。继续访问该对象，程序会得到 nil，不亏崩溃
- **unowned** 与弱引用本质上一样。不同的是，unowned 无主引用 实例销毁后仍然存储着实例的内存地址(类似于OC中的unsafe_unretained)，试图在实例销毁后访问无主引用，会产生运行时错误(野指针)
- **weak unowned** 只能用在 类实例上面
- **weak**、**unowned** 都能解决 循环引用，**unowned** 要比 **weak** 性能 稍高
- ◦ 在生命周期中可能会 变成 **nil** 的使用 **weak**
- ◦ 初始化赋值以后再也不会变成 **nil** 使用 **unowned**

4、访问控制关键字 open, public, internal, fileprivate, private 的区别？

Swift 中有个5个级别的访问控制权限,从高到低依次是 open, public, internal, fileprivate, private 它们遵循的基本规则: 高级别的变量不允许被定义为低级别变量的成员变量,比如一个 private 的 class 内部允许包含 public 的 String值,反之低级变量可以定义在高级别变量中;

- open: 具备最高访问权限,其修饰的类可以和方法,可以在任意 模块中被访问和重写。
- public: 权限仅次于 open，和 open 唯一的区别是: 不允许其他模块进行继承、重写
- internal: 默认权限, 只允许在当前的模块中访问，可以继承和重写,不允许在其他模块中访问
- fileprivate: 修饰的对象只允许在当前的文件中访问;例如它可以被一个文件中的 class，extension，struct 共同使用。
- private: 限制实体只能在其定义的作用域（类、结构体、枚举或扩展）内部访问。

访问级别	描述	同一模块内可见	不同模块内可见	示例
open	最高访问级别，允许实体被定义模块外的其他模块访问，也可以被继承和重写。	✓	✓	用于公共框架和库的公共接口，如一个开源框架的公共 API。
public	允许实体被定义模块外的其他模块访问，但不能被继承和重写。	✓	✗	用于公共接口的内部实现，如一个库的具体实现细节。
internal	默认访问级别，允许实体在定义模块内部任何地方访问，但不能被定义模块外的其他模块访问。	✓	✗	用于应用程序或框架的内部实现，如应用程序的各个组件之间的交互。
fileprivate	限制实体只能在其定义的文件内部访问。	✓	✗	用于实现细节和私有功能，如一个类的内部细节只对该类的其他成员可见。
private	限制实体只能在其定义的作用域（类、结构体、枚举或扩展）内部访问。	✓	✗	用于实现细节和私有功能，如一个结构体内部的数据成员只能在该结构体的方法中使用。

5、Swift 和OC 如何相互调用？

Swift 调用 OC代码

- 需要创建一个 Target-BriBridging-Header.h 的桥文件,在乔文件导入需要调用的OC代码[头文件](#)即可

OC 调用 Swift代码

- 直接导入 Target-Swift.h文件即可, Swift如果需要被OC调用,需要使用@objc 对方法或者属性进行修饰

6、类(class) 和 结构体(struct) 有什么区别?

在 Swift 中,class 是引用类型(指针类型), struct 是值类型

值类型

- 值类型在传递和赋值时将进行复制; 赋值给var、let或者给函数传参, 是直接将所有内容拷贝一份, 类似于对文件进行copy、paste操作, 产生了全新的文件副本。属于[深拷贝](#)(deep copy)
- 值类型: 比如结构体,枚举,是在栈空间上存储和操作的

引用类型

- 引用类型只会使用引用对象的一个"指向"; 赋值给var、let或者给函数传参, 是将内存地址拷贝一份, 类似于制作一个文件的替身(快捷方式、链接), 指向的是同一个文件。属于[浅拷贝](#)(shallow copy)
- 引用类型: 比如 Class,是在堆空间上存储和操作的

7、class 和 struct 比较,优缺点?

class 有以下功能,struct 是没有的:*

1. class可以继承,子类可以使用父类的特性和方法
2. 类型转换可以在运行时检查和解释一个实例对象
3. class可以用 deinit来释放资源
4. 一个类可以被多次引用

struct 优势:

1. 结构较小,适用于复制操作,相比较一个class 实例被多次引用,struct 更安全
2. 无需担心[内存泄露](#)问题

8、copy - on - write 或者 理解Swift中的写时复制?

值类型在复制时,复制对象 与 原对象 实际上在内存中指向同一个对象,**当且仅当 ** 修改复制的对象时,才会在内存中创建一个新的对象,

- 为了提升性能, Struct, String、Array、Dictionary、Set采取了Copy On Write的技术
- 比如仅当有“写”操作时,才会真正执行拷贝操作
- 对于标准库值类型的赋值操作, Swift 能确保最佳性能, 所有没必要为了保证最佳性能来避免赋值

原理

在结构体内部用一个引用类型来存储实际的数据,

- 在不进行写入操作的普通传递过程中, 都是将内部的reference的应用计数+1,
- 当进行写入操作时, 对内部的 reference 做一次 copy 操作用来存储新的数据; 防止和之前的 reference产生意外的数据共享。

swift中提供[[isKnownUniquelyReferenced](#)]函数, 他能检查一个类的实例是不是唯一的引用, 如果是, 我们就不需要对结构体实例进行复制, 如果不是, 说明对象被不同的结构体共享, 这时对它进行更改就需

要进行复制。

9、Swift 为什么将 Array, String, Dictionary, Set, 设计为值类型?

值类型 相比 引用类型的优点

- 值类型和引用类型相比,最大优势可以高效的使用内存;
- 值类型在栈上操作,引用类型在堆上操作;
- 栈上操作仅仅是单个指针的移动,
- 堆上操作牵涉到合并,位移,重链接

Swift 这样设计减少了堆上内存分配和回收次数,使用 copy-on-write将值传递与复制开销降到最低

String, Array, Dictionary设计成值类型, 也是为了线程安全考虑。通过Swift的let设置, 使得这些数据达到了真正意义上的“不变”, 它也从根本上解决了多线程中内存访问和操作顺序的问题

10、什么是属性观察器?

属性观察是指在当前类型内对特性属性进行监测,并作出响应,属性观察是 swift 中的特性,具有2种, `willset` 和 `didset`

- `willSet` 传递新值 `newValue`
- `didSet` 传递旧值 `oldValue`
- 在初始化器中对属性初始化时, 不会触发观察器
- 属性观察器 只能用在 存储属性, 不可用在 计算属性
- 可以为 非 `lazy` (即延迟存储属性) 的 `var` 存储属性 设置 属性观察器
- `willSet` 会传递新值, 默认叫 `newValue`
- `didSet` 会传递旧值, 默认叫 `oldValue`

注意:

- 在初始化器中设置属性值不会触发 属性观察器
 - 属性定义时设置初始值也不会出发 属性观察, 原因是
 - 属性定义时设置初始值, 本质跟在 初始化器中设置值是一样的
- 属性观察器 只能用在 存储属性, 不可用在 计算属性

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 swift
2 体验AI代码助手
3 代码解读
4 复制代码
5 struct Circle {
6     /// 存储属性
7     var radius :Double {
8         didSet {
9             print("willSet -- ",newValue,"radius == ",radius)
10        }
11        didSet {
12            print("didSet ++ ",oldValue,"radius == ",radius)
13        }
14    }
15    /*
16    上述代码 跟下面等价, 不推荐
17    var radius :Double {
18        didSet(jk_newValue) {
```

```
19         print("willSet -- ",jk_newValue,"radius == ",radius)
20     }
21     didSet(jk_oldValue) {
22         print("didSet ++ ",jk_oldValue,"radius == ",radius)
23     }
24 }
25 /*
26 */
27
28 var circle = Cicle.init(radius: 10.0)
29
30 circle.radius = 20.0
31 /*
32 willSet -- 20.0 radius == 10.0
33 didSet ++ 10.0 radius == 20.0
34 */
35
36 print("result == ",circle.radius)
37 //result == 20.0
```

拓展:

- 属性观察器, 计算属性这两个功能, 同样可以应用在全局变量/局部变量
- 属性观察器, 计算属性 不可以同时 应用在同一个类(不包括继承)的属性中

11、Swift 异常捕获

do - try - catch 机制

- Swift中可以通过 `Error` 协议自定义运行时的错误信息
- 函数内部通过 `throw` 抛出 自定义 `Error` , 抛出 `Error` 的函数必须加上 `throws` 声明(逻辑通过不会抛出, 反之可能抛出)
- 需要使用 `try` 调用 可能会 抛出的 `Error` 函数
- 通过 `try` 尝试调用 函数 抛出的异常 必须要 处理异常; 否在会编译报错; 反之 运行时 在 `top level (main)` 报错, 闪退

12、defer 的用法

- 使用defer代码块来表示在函数返回前, 函数中最后执行的代码。无论函数是否会抛出错误, 这段代码都将执行。

13、如何将Swift中协议 部分方法 设计成可选?

- 方案一(不推荐, 除非需要暴露给OC用)
- 在协议和方法前面添加 `@objc` , 然后在方法前面添加 `optional` 关键字, 改方式实际上是将协议转为了OC的方式

swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 swift
2 体验AI代码助手
3 代码解读
4 复制代码
5 @objc protocol someProtocol {
6     @objc optional func test()
7 }
```

协议 可以用来定义 **属性 方法 下标** 的 **声明**,协议 可以被 **类 结构体 枚举** 遵守 (多个协议之间用 **,** 隔开)

▼ swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 swift
2 体验AI代码助手
3 代码解读
4 复制代码
5 protocol Drawable {
6     func draw()
7     var x:Int { get set }
8     var y:Int { get }
9     subscript(index:Int) -> Int {get set}
10 }
11
12 protocol Test1 {}
13 protocol Test2 {}
14 protocol Test3 {}
15
16 class TestClass: Test1,Test2,Test3 {
17
18 }
```

注意:

- 协议中定义的方法, 不能有默认参数
- 默认情况下, 协议中定义的内容需要全部实现

14、Swift和OC中的 protocol 有什么不同?

- 相同点, 两者都是用作代理
- 不同点
 - ◦ Swift
 - Swift中的 protocol 还可以对接口进行抽象,可以实现面向协议,从而大大提高编程效率
 - Swift中的 protocol 可以用于值类型,结构体,枚举;

15、比较Swift 和OC中的初始化方法 (init) 有什么不同?

swift 的初始化方法,因为引入了[两段式初始化和安全检查](#)因此更加严格和准确,

swift初始化方法需要保证所有的非optional的成员变量都完成初始化,

同时 swfit 新增了convenience和 required两个修饰初始化器的关键字

- convenience只提供一种方便的初始化器,必须通过一个指定初始化器来完成初始化
- required是强制子类重写父类中所修饰的初始化方法

16、Swift 和OC 中的自省 有什么区别?

- OC
 - ◦ 自省在OC中就是判断某一对象是否属于某一个类的操作,有以下2中方式
- ◦
 - `[obj isKindOfClass:[SomeClass class]]` : obj 必须是 SomeClass 的 对象或 其子类对

```
    象;return YES;
```

- `[obj isKindOfClass:[SomeClass class]]` : obj 必须是 SomeClass 的对象;return YES;

- Swift

- - Swift 中由于很多 class 并非继承自 NSObject, 故而 Swift 使用 `is` 来判断是否属于某一类型, is 不仅可以作用于class, 还可以作用于enum和struct

17、什么是函数重载? swift 支不支持函数重载?

- 函数重载是指: 函数名称相同,函数的参数个数不同, 或者参数类型不同,或参数标签不同, 返回值类型与函数重载无关
- swift 支持函数重载

构成函数重载的规则

- 函数名相同
- 参数个数不同 || 参数类型不同 || 参数标签不同
-

注意: 返回值类型 与函数重载无关; 返回值类型不同时, 函数重载会报错:

▼ Swift

 体验AI代码助手  代码解读 复制代码

```
1  swift
2  体验AI代码助手
3  代码解读
4  复制代码
5  func overloadsum(v1 : Int,v2:Int) -> Int {
6      v2 + v1
7  }
8
9  // 参数个数不同
10 func overloadsum(v1 : Int,v2:Int,v3:Int) -> Int {
11     v2 + v1 + v3
12 }
13
14 // 参数类型不同
15 func overloadsum(v1 : Int,v2:Double) -> Double {
16     v2 + Double(v1)
17 }
18
19 // 参数标签不同
20 func overloadsum(_ v1 : Int,_ v2:Int) -> Int {
21     v2 + v1
22 }
23
24 func overloadsum(a : Int,_ b:Int) -> Int {
25     a + b
26 }
27
28 /**
29  返回值类型不同时, 在函数重载时, 会报错:
30  Ambiguous use of 'overloadsum(v1:v2:)'
31
32  func overloadsum(v1 : Int,v2:Int) {
33  }
34  */
35
36 public func overloadtest() {
37     let result1 = overloadsum(v1: 10, v2: 20)
38     let result2 = overloadsum(v1: 10, v2: 20, v3: 30)
39     let result3 = overloadsum(v1: 10, v2: 20)
40     let result4 = overloadsum(10, 20)
41     let resutt4_1 = overloadsum(a: 10, 20)
```

```
42
43     print(result1,result2,result3,result4,resutt4_1)
44     //30 60 30 30 30
45 }
```

18、闭包是引用类型吗？

闭包和函数都是是引用类型。如果一个闭包被分配给一个变量，这个变量复制给另一个变量，那么他们引用的是同一个闭包，他们的捕捉列表也会被复制。

swift 中的闭包结构是什么样子的？

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 markdown
2     体验AI代码助手
3     代码解读
4     复制代码
5 {
6     (参数列表) -> 返回值类型 in 函数体代码
7 }
8
```

什么是尾随闭包

- 尾随闭包 是一个被 书写在 函数调用 括号 后面的 闭包表达式

基本定义

●Swift中可通过 func 定义一个函数，也可以通过 闭包表达式 定义一个函数

闭包表达式的格式：

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 markdown
2     体验AI代码助手
3     代码解读
4     复制代码
5 {
6     (参数列表) -> 返回值类型 in
7     函数体代码
8 }
9
```

闭包表达式与定义函数的语法相对比，有区别如下：

- 1没有func
- 2没有函数名
- 3返回值类型添加了关键字in

▼ kotlin  体验AI代码助手  代码解读 复制代码

```
1 kotlin
2     体验AI代码助手
3     代码解读
4     复制代码
5 let fn1 = {
6     (v1 : Int,v2 : Int) -> Int in
7     return v1 + v2
8 }
9
```

```
10 let result1 = fn1(10,5)
11
12 let result2 = {
13     (v1:Int,v2:Int) -> Int in
14     return v2 + v1
15 }(10,6)
16
17 print(result1,result2) // 15 16
```

闭包表达式的简写

▼ php  体验AI代码助手  代码解读 复制代码

```
1 php
2 体验AI代码助手
3 代码解读
4 复制代码
5 func exec(v1:Int,v2:Int,fn:(Int,Int)->Int) {
6     print(fn(v1,v2))
7 }
8
9 闭包表达式的简写
10 private func test2() {
11     // 1: 没有简写
12     exec(v1:10, v2:20, fn: {
13         (v1:Int,v2:Int) -> Int in
14         return v1 + v2
15     })
16
17     // 2: 简写1
18     exec(v1: 2, v2: 3, fn: {
19         v1,v2 in return v1 + v2
20     })
21
22     // 3:简写 2
23     exec(v1: 3, v2: 4, fn: {
24         v1,v2 in v1 + v2
25     })
26
27     // 4: 简写3
28     exec(v1: 5, v2: 6, fn: {$0 + $1})
29
30     // 5: 简写4
31     exec(v1: 7, v2: 8, fn: +)
32 }
```

尾随闭包

- 将一个很长的闭包表达式作为函数的最后一个实参, 使用尾随闭包可以增强代码的可读性
- ◦ 尾随闭包 是一个被 书写在 函数调用 括号 后面的 闭包表达式

▼ javascript  体验AI代码助手  代码解读 复制代码

```
1 javascript
2 体验AI代码助手
3 代码解读
4 复制代码
5 func test3() {
6
7     exec(v1: 8, v2: 7) { a, b in
8         a + b
9     }
10
11     // or      { 书写在 函数调用 括号 后面的 闭包表达式}
12     exec(v1: 9, v2: 10) {
```

```
13         $0 + $1
14     }
15 }
```

- 如果 闭包表达式 是函数的唯一 实参, 且使用了尾随闭包的 语法, 则在函数名后面的 `()` 可省略

```
javascript 体验AI代码助手 代码解读 复制代码

1 javascript
2   体验AI代码助手
3   代码解读
4   复制代码
5   // fn: 就是尾随闭包
6   func exec1(fn:(Int,Int)->Int) {
7       print(fn(1,2))
8   }
9
10  exec1(fn: {$0 + $1})
11  exec1() {$0 + $1}
12  exec1{$0 + $1}
```

什么是逃逸闭包

- 闭包有可能在函数结束后调用, 闭包调用 逃离了函数的作用域, 需要通过 `@escaping` 声明

注意: 逃逸闭包 不可以 捕获 inout 参数

原因是: 逃逸闭包不确定 何时开始执行, 有可能 在执行逃逸闭包时, 可变参数已经被程序回收, 造成野指针访问

什么是自动闭包

自动闭包是一种自动创建的用来把作为实际参数传递给函数的表达式打包的闭包。

它不接受任何实际参数, 并且当它被调用时, 它会返回内部打包的表达式的值。

这个语法的好处在于通过写普通表达式代替显式闭包而使你省略包围函数形式参数的括号。

```
less 体验AI代码助手 代码解读 复制代码

1 less
2   体验AI代码助手
3   代码解读
4   复制代码
5   func getFirstPositive(_ v1: Int, _ v2: @autoclosure () -> Int) -> Int? {
6       return v1 > 0 ? v1 : v2()
7   }
8   getFirstPositive(10, 20)
```

- 为了避免与期望冲突, 使用了@autoclosure的地方最好明确注释清楚:这个值会被推迟执行
- @autoclosure 会自动将 20 封装成闭包 { 20 }
- @autoclosure 只支持 () -> T 格式的参数
- @autoclosure 并非只支持最后1个参数
- 有@autoclosure、无@autoclosure, 构成了函数重载

如果你想要自动闭包允许逃逸, 就同时使用 @autoclosure 和 @escaping 标志。

19、Swift 中的存储属性与计算属性

存储属性(Stored Property)

- 类似于成员变量这个概念
- 存储在实例对象的内存中
- 结构体、类可以定义存储属性
- 枚举不可以定义存储属性

关于 存储属性, Swift 中有个明确的规定

- 在创建 类 或者 结构体 的实例时, 必须为所有的存储属性设置一个合适的初始值
 - 可以在初始化器里为存储属性设置一个初始值
 - 可以分配一个默认的属性值作为属性定义的一部分

计算属性(Computed Property)

- 本质就是方法(函数)
- 不占用实例对象的内存
- 枚举、结构体、类都可以定义计算属性

计算属性(Computed Property)

○本质就是方法(函数)

○不占用实例的内存

○枚举、结构体、类 都可以定义计算属性

理解计算属性与存储属性:

如果两个属性之间存在一定的逻辑关系, 使用计算属性, 原因如下:

- 如果都用存储属性的话, 其逻辑对应关系可能有误
- 而使用计算属性, 则可以准确的描述 这种逻辑关系

具体案例参考 下面的 Cicle 中的 radius(半径) 和 diameter(直径) 的逻辑关系

同时因为计算属性不占用实例的内存, 可以有效的节省实例的内存空间

- set 传入的新值 默认叫做 newValue,也可以自定义

▼ kotlin

 体验AI代码助手  代码解读 复制代码

```
1 kotlin
2 体验AI代码助手
3 代码解读
4 复制代码
5 struct Cicle {
6     /// 存储属性
7     var radius :Double
8     /// 计算属性
9     var diameter: Double {
10         get {
11             radius * 2
12         }
13         set (jkNewValue){
14             radius = jkNewValue / 2.0
15         }
16     }
17 }
```

- 只读计算属性: 只有 `get` , 没有 `set`

▼ swift

 体验AI代码助手  代码解读 复制代码

```
1 swift
```

```
2  体验AI代码助手
3  代码解读
4  复制代码
5  struct Circle {
6      /// 存储属性
7      var radius :Double
8      /// 计算属性
9      /*
10     var diameter: Double {
11         get {
12             radius * 2
13         }
14     }
15     */
16
17     // 上述代码与下面的代码等价
18     var diameter: Double {radius * 2}
19 }
20
21 var circle = Circle.init(radius: 12)
22
23 print(circle.radius)//12.0
24 print(circle.diameter)//24.0
25
26 // circle.diameter = 10.0 //Cannot assign to property: 'diameter' is a get-only property
```

- 定义计算属性只能用 var,不可以是 let
- - let 表示常量
 - 计算属性的值是可能会发生变化的(包括只读计算属性)

20、什么是[延迟存储属性](Lazy Stored Property)

- 使用 lazy 可以定义一个 延迟存储属性, 在第一次用到属性的时候才会进行初始化(类似 OC 中的懒加载)

注意点:

- lazy 属性 必须是 var
- - 因为, let 属性 必须在实例的初始化方法 完成之前 就拥有值
- 如果多条线程同时第一次访问 lazy 属性, 无法保证 属性 只被 初始化 一次 (即线程不是安全的)
- 使用 lazy 可以定义一个 延迟存储属性, 在第一次用到属性的时候才会进行初始化(类似 OC 中的懒加载)

注意点:

- lazy 属性 必须是 var
- - 因为Swift 规定: let 属性 必须在实例的初始化方法 完成之前 就拥有值
- 如果多条线程同时第一次访问 lazy 属性, 无法保证 属性 只被 初始化 一次 (即线程不是安全的)

```
swift
体验AI代码助手 代码解读 复制代码

1 swift
2 体验AI代码助手
3 代码解读
4 复制代码
5 class Car {
6     init() {
7         print("car has init")
8     }
9 }
```

```
10 func run() {
11     print("car running")
12 }
13 }
14
15 class Person {
16     lazy var car = Car.init()
17
18     init() {
19         print("person has init")
20     }
21
22     func go_out() {
23         car.run()
24     }
25 }
26
27 let p = Person.init() //person has init
28 print("*****")
29 p.go_out()// car has init ----> car running
```

- 当结构体 包含 一个延迟存储属性时, 只有 var 才能访问延迟 存储属性
- - 因为延迟属性 初始化时 要改变 结构体的内存

▼ swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 swift
2 体验AI代码助手
3 代码解读
4 复制代码
5 struct Point {
6     var x = 0
7     var y = 0
8     lazy var z = 0
9 }
10
11 let p = Point.init()
12 print(p.z)//Cannot use mutating getter on immutable value: 'p' is a 'let' constant
```

21、什么是 类型 属性?

- 类型属性(Type Property) :通过类型去访问
- - 存储类型属性(Stored Type Property):整个程序运行过程中, 就只有1份内存(类似于全局变量, 底层采用了 gcd_once 操作)
 - 计算类型数据(Computed Type Property):

属性可分为

●实例属性(Instance Property):通过实例去访问

○存储实例属性(Stored Instance Property):存储在实例的内存中, 每个实例都有1分

○计算实例属性(Computed Instance Property): 不占用实例的内存, 本质是方法

●类型属性(Type Property) :通过类型去访问

○存储类型属性(Stored Type Property):整个程序运行过程中, 就只有1份内存(类似于全局变量, 底层采用了 gcd_once 操作, 保证只初始化一次)

○计算类型数据(Computed Type Property):

注意:

存储类型属性不会 占用 实例对象 的内存, 整个程序运行过程中, 就只有1份内存

存储类型属性 本质就是全局变量(该全局变量加了一些类型控制, 只能通过类型去访问),

可以通过 `static` 定义类型属性

如果 是类, 也可以使用 关键字 `class`

结构体 就只能使用 关键字 `static`

swift

 体验AI代码助手  代码解读 复制代码

```
1 swift
2 复制代码
3 struct Car {
4     static var count:Int = 0
5     init(){
6         Car.count += 1
7     }
8 }
9
10 let c1 = Car.init()
11 let c2 = Car.init()
12 let c3 = Car.init()
13 print(Car.count)//3
```

类型属性的细节:

- 不同于 存储实例属性, 必须给 存储类型属性 设定初始值
- 因为类型没有想实例那样的 `init` 初始化器来初始化 存储属性
- 存储类型属性 默认就是 `lazy`。会在第一次使用的时候 初始化
- 就算 被 多个线程 同时访问, 保证只会被 初始化 一次, 线程是安全的
- 存储类型 属性 也可以是 `let`
- 枚举类型 也可以 定义 类型属性(存储类型属性, 计算类型属性)

22、Swift 中如何定义单例模式

可以通过 类型属性+`let+private` 来写单例; 代码如下如下:

csharp

 体验AI代码助手  代码解读 复制代码

```
1 csharp
2 体验AI代码助手
3 代码解读
4 复制代码
5 // 方式一
6 public class SingleManager{
7     public static let shared = SingleManager()
8     private init(){}
9 }
10
11 // 方式二
12 public class SingleManager{
13     public static let shared = {
14         //...
15         //...
16         return SingleManager()
17     }()
18     private init(){}
19 }
20
21 // 上述两个方法等价, 一般推荐 方式二
```

23、什么是运算符重载?

类、结构体、枚举可以为现有的运算符提供自定义的实现, 这个操作叫做:运算符重载

less

体验AI代码助手

代码解读

复制代码

```
1 less
2 体验AI代码助手
3 代码解读
4 复制代码
5 struct Point {
6     var x: Int,y: Int
7     static func + (p1: Point,p2: Point) -> Point {
8         Point(x: p1.x + p2.x ,y: p1.y + p2.y)
9     }
10 }
11 let p = Point(x:10,y: 20) + Point(x: 30,y: 40)
12 print(p)
```

24、平时用过哪些Swift的高阶函数？

map：给定数组中的取出值，对其应用一个函数，再将函数的结果返回到新容器中

compactMap：与map函数的功能一致，它会从结果的集合中过滤掉nil，解包可选值

flatMap：与map函数的功能一致，它会打平结果集合（比如二维数组会变成一个新的一维数组）

map

map 函数用于对集合中的每个元素应用一个指定的转换闭包，然后返回一个包含转换结果的新集合。函数定义：

swift

体验AI代码助手

代码解读

复制代码

```
1 func map<T>(<_ transform: (Element) throws -> T) rethrows -> [T]
```

示例：

ini

体验AI代码助手

代码解读

复制代码

```
1 let numbers = [1, 2, 3, 4, 5]
2 let doubled = numbers.map { $0 * 2 }
3 print(doubled) // 输出 [2, 4, 6, 8, 10]
```

filter

filter 函数用于从集合中选择满足指定条件的元素，并返回一个包含满足条件的元素的新集合。函数定义：

swift

体验AI代码助手

代码解读

复制代码

```
1 func filter(<_ isIncluded: (Element) throws -> Bool) rethrows -> [Element]
```

示例：

ini

体验AI代码助手

代码解读

复制代码

```
1 let numbers = [1, 2, 3, 4, 5]
2 let evenNumbers = numbers.filter { $0 % 2 == 0 }
3 print(evenNumbers) // 输出 [2, 4]
```

reduce

reduce 函数用于将集合中的所有元素组合成单个值，并返回该值。函数定义：

▼ swift  体验AI代码助手  代码解读 复制代码

```
1 func reduce<Result>(_ initialResult: Result, _ nextPartialResult: (Result, Element) throws -> Result)
```

示例：

▼ bash  体验AI代码助手  代码解读 复制代码

```
1 let numbers = [1, 2, 3, 4, 5]
2 let sum = numbers.reduce(0, { $0 + $1 })
3 print(sum) // 输出 15
```

flatMap

flatMap 函数用于对集合中的每个元素应用一个转换闭包，并将结果拼接成一个新的集合。函数定义：

▼ swift  体验AI代码助手  代码解读 复制代码

```
1 func flatMap<SegmentOfResult>(_ transform: (Element) throws -> SegmentOfResult) rethrows -> [SegmentOfResult]
```

示例：

▼ ini  体验AI代码助手  代码解读 复制代码

```
1 let nestedArray = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2 let flatArray = nestedArray.flatMap { $0 }
3 print(flatArray) // 输出 [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

compactMap

compactMap 函数用于对集合中的每个元素应用一个转换闭包，并过滤掉结果中的 nil 值。函数定义：

▼ swift  体验AI代码助手  代码解读 复制代码

```
1 func compactMap<ElementOfResult>(_ transform: (Element) throws -> ElementOfResult?) rethrows -> [ElementOfResult]
```

示例：

▼ ini  体验AI代码助手  代码解读 复制代码

```
1 let strings = ["1", "2", "3", "hello", "5"]
2 let numbers = strings.compactMap { Int($0) }
3 print(numbers) // 输出 [1, 2, 3, 5]
```

allSatisfy

allSatisfy 函数用于检查序列中的所有元素是否都满足指定条件。函数定义：

▼ swift  体验AI代码助手  代码解读 复制代码

```
1 func allSatisfy(_ predicate: (Element) throws -> Bool) rethrows -> Bool
```

示例:

▼ bash  体验AI代码助手  代码解读 复制代码

```
1 let numbers = [2, 4, 6, 8, 10]
2 let allEven = numbers.allSatisfy { $0 % 2 == 0 }
3 print(allEven) // 输出 true
```

25、Swift final 关键字 用过没 主要用来干嘛

final 关键字在 Swift 中用于限制类、属性和方法的继承和重写。具体来说，final 有以下用途：

限制类的继承：通过在类的定义前加上 final 关键字，可以防止其他类继承该类。这样做可以确保类的实现不会被修改或扩展，从而提高代码的安全性和稳定性。

▼ kotlin  体验AI代码助手  代码解读 复制代码

```
1 final class FinalClass {}
2 // OtherClass: FinalClass // 编译错误, FinalClass 不能被继承
```

限制属性的重写：通过在属性的定义前加上 final 关键字，可以防止子类重写该属性。这样做可以确保父类的属性在子类中保持不变。

▼ kotlin  体验AI代码助手  代码解读 复制代码

```
1 class ParentClass {
2     final var finalProperty: Int = 42
3 }
4
5 class ChildClass: ParentClass {
6     // override var finalProperty: Int // 编译错误, finalProperty 不能被重写
7 }
```

限制方法的重写：通过在方法的定义前加上 final 关键字，可以防止子类重写该方法。这样做可以确保父类的方法在子类中保持不变。

▼ kotlin  体验AI代码助手  代码解读 复制代码

```
1 class ParentClass {
2     final func finalMethod() {
3         print("ParentClass final method")
4     }
5 }
6
7 class ChildClass: ParentClass {
8     // override func finalMethod() { // 编译错误, finalMethod 不能被重写
9     //     print("ChildClass override method")
10    // }
11 }
```

总之，final 关键字用于限制类、属性和方法的继承和重写，从而提高代码的安全性和稳定性。特别是在框架或库中希望确保类的行为不被修改时非常有用。

26、Swift 的 extension 用过没，都能干嘛

是的，extension 在 Swift 中经常被使用。Swift 的 extension（扩展）允许你在不修改原始代码的情况下，扩展现有类型的功能，包括类、结构体、枚举和协议。使用 extension 可以实现以下功能：

添加新的方法：可以在已有的类型上添加新的实例方法或类型方法。

```
swift 体验AI代码助手 代码解读 复制代码

1 extension Int {
2     func squared() -> Int {
3         return self * self
4     }
5 }
6
7 let number = 5
8 let squaredNumber = number.squared() // 25
```

添加新的计算属性：可以在已有的类型上添加新的计算属性。

```
swift 体验AI代码助手 代码解读 复制代码

1 extension Double {
2     var squared: Double {
3         return self * self
4     }
5 }
6
7 let doubleNumber = 3.0
8 let squaredDouble = doubleNumber.squared // 9.0
```

定义新的初始化方法：可以在已有的类型上定义新的初始化方法。

```
php 体验AI代码助手 代码解读 复制代码

1 extension String {
2     init(repeating character: Character, count: Int) {
3         var string = ""
4         for _ in 0..
```

遵循协议：可以使已有的类型遵循新的协议。

```
swift 体验AI代码助手 代码解读 复制代码

1 protocol CustomProtocol {
2     func customMethod()
3 }
4
5 extension Int: CustomProtocol {
6     func customMethod() {
7         print("Custom method called on Int")
8     }
9 }
10
11 let number = 10
12 number.customMethod() // 输出: Custom method called on Int
```

提供默认实现：可以在协议的扩展中为协议中的方法提供默认实现。

```
swift 体验AI代码助手 代码解读 复制代码
```

```
1 protocol SomeProtocol {
2     func requiredMethod()
3 }
4
5 extension SomeProtocol {
6     func requiredMethod() {
7         print("Default implementation of required method")
8     }
9 }
10
11 struct SomeStruct: SomeProtocol {}
12
13 let someStruct = SomeStruct()
14 someStruct.requiredMethod() // 输出: Default implementation of required method
```

总之, extension 是一种强大的工具, 用于扩展已有类型的功能, 使代码更加模块化、易于维护和扩展。

27、简要说明Swift中的初始化器？

- 结构体会默认生成 含有参数的初始化器,一旦自定义初始化器, 默认的初始化器则不可用
- 类默认只会生成无参的指定初始化器
- 类、结构体、枚举都可以定义初始化器
- 类有2种初始化器: 指定初始化器(designated initializer)、便捷初始化器(convenience initializer)

▼ swift [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 swift
2 代码解读
3 复制代码
4 // 指定初始化器
5 init(parameters) {
6     statements
7 }
8 // 便捷初始化器
9 convenience init(parameters) {
10     statements
11 }
```

规则:

- 每个类至少有一个指定初始化器, 指定初始化器是类的主要初始化器
- 默认初始化器总是类的指定初始化器
- 类偏向于少量指定初始化器, 一个类通常只有一个指定初始化器

初始化器的相互调用规则

- 指定初始化器必须从它的直系父类调用指定初始化器
- 便捷初始化器必须从相同的类里调用另一个初始化器
- 便捷初始化器最终必须调用一个指定初始化器

28、swift 中的枚举,关联值 和 原始值的区分？

▼ sql [体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 sql
2 代码解读
```

```
3 复制代码
4 答：
5 1.关联值——有时会将枚举的成员值跟其他类型的变量关联存储在一起，存储在枚举变量中，占用枚举变量内存
6 // 关联值
7 enum Date {
8     case digit(year: Int, month: Int, day: Int)
9     case string(String)
10 }
11
12 2.原始值——枚举成员可以使用相同类型的默认值预先关联，这个默认值叫做：原始值，不会存储在 枚举变量中，不占用枚举变量内存
13 // 原始值
14 enum Grade: String {
15     case perfect = "A"
16     case great = "B"
17     case good = "C"
18     case bad = "D"
19 }
20
21 复制代码
```

29、swift中,关键字 guard 和 defer 的用法

▼ swift

[体验AI代码助手](#) [代码解读](#) [复制代码](#)

```
1 swift
2 代码解读
3 复制代码
4 答：
5 guard也是基于一个表达式的布尔值去判断一段代码是否该被执行。与if语句不同的是，guard只有在条件不满足的时候才会执行这段
6 guard let name = self.text else { return }
7
8
9 defer的用法是，这条语句并不会马上执行，而是被推入栈中，直到函数结束时才再次被调用。
10 defer {
11     // 函数结束才调用
12 }
13 复制代码
```

30、比较Swift 和OC中的初始化方法 (init) 有什么不同？

swift 的初始化方法,更加严格和准确, swift初始化方法需要保证所有的非optional的成员变量都完成初始化, 同时 swift 新增了convenience和 required两个修饰初始化器的关键字

convenience只提供一种方便的初始化器,必须通过一个指定初始化器来完成初始化。

required是强制子类重写父类中所修饰的初始化方法

30、Swift中strong 、weak和unowned是什么意思？二者有什么不同？何时使用unowned？

Swift 的内存管理机制与 Objective-C一样为 ARC（Automatic Reference Counting）。它的基本原理是，一个对象在没有任何强引用指向它时，其占用的内存会被回收。反之，只要有任何一个强引用指向该对象，它就会一直存在于内存中。

- strong 代表着强引用，是默认属性。当一个对象被声明为 strong 时，就表示父层级对该对象有一个强引用的指向。此时该对象的引用计数会增加1。
- weak 代表着弱引用。当对象被声明为 weak 时，父层级对此对象没有指向，该对象的引用计数不会

增加1。它在对象释放后弱引用也随即消失。继续访问该对象, 程序会得到 nil, 不亏崩溃

- **unowned** 与弱引用本质上一样。不同的是, unowned 无主引用 实例销毁后仍然存储着实例的内存地址(类似于OC中的unsafe_unretained), 试图在实例销毁后访问无主引用, 会产生运行时错误(野指针)
- **weak unowned** 只能用在 类实例上面
- **weak**、**unowned** 都能解决 循环引用, **unowned** 要比 **weak** 性能 稍高
- - 在生命周期中可能会 变成 **nil** 的使用 **weak**
 - 初始化赋值以后再也不会变成 **nil** 使用 **unowned**

31、Swift 和OC 中的自省 有什么区别?

- OC
 - 自省在OC中就是判断某一对象是否属于某一个类的操作,有以下2中方式
 - - **[obj isKindOfClass:[SomeClass class]]** : obj 必须是 SomeClass 的 对象或 其子类对象;return YES;
 - **[obj isKindOfClass:[SomeClass class]]** : obj 必须是 SomeClass 的 对象;return YES;
- Swift
 - Swift 中由于很多 class 并非继承自 NSObject, 故而 Swift 使用 **is** 来判断是否属于某一类型, is 不仅可以作用于class, 还可以作用于enum和struct

32、Swift定义常量 和 OC定义常量的区别?

cssharp  体验AI代码助手  代码解读 复制代码

```
1
2 //OC:
3 const int price = 0;
4 //Swift:
5 let price = 0
```

- OC中 const 常量类型和数值是在编译时确定的
- Swift 中 let 常量 (只能赋值一次),其类型和值既可以是静态的, 也可以是一个动态的计算方法, 它们在运行时确定的。

33、Swift中函数的柯里化

将一个 接受 多个参数的 函数, 变成 只接受 一个参数的一系列 操作

示例

swift  体验AI代码助手  代码解读 复制代码

```
1 swift
2 代码解读
3 复制代码
4 func add(_ v1: Int,_ v2: Int) -> Int {
5     v1 + v2
6 }
7
8 func difference(_ v1: Int,_ v2: Int) -> Int {
9     v1 - v2
10 }
11
12 func add2(_ v1: Int,_ v2: Int,_ v3 :Int ,_ v4 :Int) -> Int {
13     v1 + v2 - v3 + v4
14 }
```

- 伪柯里化

swift

体验AI代码助手 代码解读 复制代码

```
1 swift
2 代码解读
3 复制代码
4 func currying_add(_ v1:Int) -> (Int) -> Int {
5     return {$0 + v1}
6 }
7
8 /*
9 将任意一个 接受两个 参数的函数 柯里化
10 */
11 func currying_fun_tow_params1(_ f1 :@escaping (Int,Int) -> Int, _ v1 :Int) -> (Int) -> Int {
12     // return {
13     //     f1($0,v1)
14     // }
15     return { (v2) in
16         return f1(v1 , v2)
17     }
18 }
19
20 print(add(10, 20)) // 30
21 print(currying_add(10)(20)) // 30 // 30
22 print(currying_fun_tow_params1(add, 10)(20))
```

- 正宗柯里化

less

体验AI代码助手 代码解读 复制代码

```
1 less
2 代码解读
3 复制代码
4 func currying_fun_two_params2<A,B,C>(_ f1: @escaping (A,B) -> C) -> (A) -> ((B) -> C) {
5     /*
6     return {
7         (a) in // a = 3
8         return {
9             (b) in // b = 8
10             return f1(a,b)
11         }
12     }
13     */
14
15     { a in { b in f1(a, b)} }
16
17 }
18
19 let result = currying_fun_two_params2(add)(3)(5)
20 print("result == ",result) //8
```

- 柯里化拓展

scss

体验AI代码助手 代码解读 复制代码

```
1 scss
2 代码解读
3 复制代码
4
5 /*
6 -> (A) -> (B) -> (C) -> (D) -> E
7
8 实际是 一连串 闭包的 组合 如下所示:
9
10 -> (A) -> ( (B) -> ((C) -> ((D) -> E)) )
11
12
```

```
13 传入 A  > 一个 闭包  (B)  ->  ( (C) -> ((D) -> E) )
14
15
16 传入 B  >> 一个 闭包  (C)  ->  ( (D) -> E )
17
18
19 传入 C  >> 一个闭包  (D) -> E
20
21
22 */
23
24
25 //func currying_fun_more_params<A,B,C,D,E>(_ fn: @escaping (A,B,C,D) -> (E)) -> (A) -> ((B) -> ((C) -
26 func currying_fun_more_params<A,B,C,D,E>(_ fn: @escaping (A,B,C,D) -> (E)) -> (A) -> (B) -> (C) ->
27 /*
28     return {
29         (a) in
30         return {
31             (b) in
32             return {
33                 (c) in
34                 return {
35                     d in
36                     return fn(a,b,c,d)
37                 }
38             }
39         }
40     }
41 */
42
43 {a in { b in { c in { d in fn(a,b,c,d)}}}}
44 }
45
46 let resutl2 = currying_fun_more_params(add2)(10)(20)(30)(40)
47 print(resutl2) // 40
48
49 let resutl2_func = currying_fun_more_params(add2)
50 let resutl2_func_value = resutl2_func(10)(20)(30)(40)
51 print(resutl2_func_value) // 40
```

标签： 面试 Swift 话题： 我的技术写作成长之路

本文收录于以下专栏

 **Swift** 专栏目录

Swift Swift相关技术

0 订阅 · 2 篇文章

[订阅](#)

[下一篇](#) [一文了解-移动开发中的三方库开源协议](#)

评论 0

抢首评，友善交流



0 / 1000

发送

暂无评论数据

为你推荐

- 2021年7月最新iOS面试题总结（答案篇）

iOS南方 3年前 9.0k 14 1 面试
- Java面试题及答案整理【最新】（腾讯、阿里、字节跳动、百度、美团）

不秃头的程序员丶 3年前 2.1k 1 2 Java
- 程序员找工作必备 PHP 基础面试题 （三）

八重樱 5年前 480 5 评论 PHP
- 2020年大厂Java面试前复习的正确姿势（800+面试题附答案解析）

程序员追风 5年前 76k 581 17 Java 面试
- TP-LINK提前批（图像算法岗）6月7日-6月8日 面试题6道

七月在线 3年前 485 点赞 评论 面试
- DBA面试资源合集（含Oracle、MySQL、Redis等）

墨天轮 3年前 2.2k 3 评论 数据库 面试
- 2019Flutter面试题最新整理大全（含答案）

Android高级开发工... 5年前 960 1 评论 Android
- 2020年Java面试题汇总篇——附答案

Java中文社群 5年前 7.6k 50 评论 Java
- 2023 年最新互联网 Java 面试八股文出炉（附大厂 P5-P8 技术栈）

董嘉dongjia 2年前 397 点赞 评论 后端
- PHP高级程序员必看知识点：目录大全（不定期更新）

八重樱 5年前 2.0k 6 评论 PHP
- 程序员找工作必备 PHP 基础面试题 （四）

八重樱 5年前 516 5 评论 PHP
- iOS经典面试题大全2021（一）（附答案）

iOS南方 3年前 1.6k 4 评论 面试
- iOS高级开发面试题合集（一）

iOS小王 3年前 6.1k 23 1 面试 iOS
- JS 经典面试题初篇(this, 闭包, 原型...)含答案

LinYiYi 4年前 11k 126 20 JavaSc...



Java中级面试题及答案大全（2021年Java面试题答案大汇总）

架构师专栏

3年前

👁 2.7k

👍 3

💬 评论

面试

Java

助手

